

WEEK 4 — ML SYSTEM DESIGN DOCUMENT

HealthConnect Appointment No-Show Prediction System

Track: Machine Learning Engineering

Project: HealthConnect Experience Lab

Week: 4

Objective: Initial ML System Design and Implementation Planning

1. ML Problem Overview

1.1 Background

HealthConnect manages patient appointments where some scheduled appointments are not attended by patients. Patient no-shows can negatively affect healthcare service delivery by creating unused appointment slots, increasing waiting times, reducing operational efficiency, and making resource planning more difficult.

The objective of this machine learning project is to develop a predictive system that estimates the likelihood that a scheduled appointment will result in a **No-Show**.

The prediction can help HealthConnect identify appointments that may require additional attention, such as reminders or other appropriate interventions.

1.2 Machine Learning Problem

The problem will be treated as a **supervised classification problem**.

The initial prediction task will be:

Given information available about a scheduled appointment, predict whether the patient will attend or fail to attend the appointment.

For the initial binary modelling approach:

- **1 = No-Show**
- **0 = Attended**

Cancelled appointments will require separate consideration during data preparation because they represent a different outcome from a patient simply failing to attend.

1.3 Business Objective

The ML system should help HealthConnect:

- Identify appointments with a high probability of no-show.
- Support targeted appointment-management interventions.
- Improve appointment utilization.
- Reduce avoidable unused appointment slots.
- Support better operational planning.

The model should be treated as a **decision-support tool**, rather than an automatic system for making decisions about patients.

2. Expected Input and Output

2.1 Input

The prediction system will receive information about an upcoming appointment.

Potential input features include:

Feature	Description
gender	Recorded patient gender
age	Patient age
age_group	Age category
appointment_type	Type of appointment
appointment_day	Day of the scheduled appointment

appointment time	Morning, afternoon or evening
booking_lead_days	Number of days between booking and appointment
previous_appointments	Number of previous appointments
previous_no_shows	Number of previous missed appointments
reminder_sent	Whether a reminder was sent
reminder_channel	SMS, WhatsApp, Email, etc.
distance_to_clinic_km	Approximate distance to clinic
waiting time minutes	Estimated waiting time

The identifiers appointment_id and patient_id should generally **not be used as predictive features**, because they identify records rather than represent meaningful patient behaviour.

Important data-leakage consideration

The system should only use information that would genuinely be available **before the appointment occurs**. For example, information generated after an appointment should not be used to make that appointment's prediction.

2.2 Output

The system should produce:

Primary output

Predicted appointment outcome

Example:

Prediction: No-Show

or

Prediction: Attended

Secondary output

The model should ideally also provide a probability:

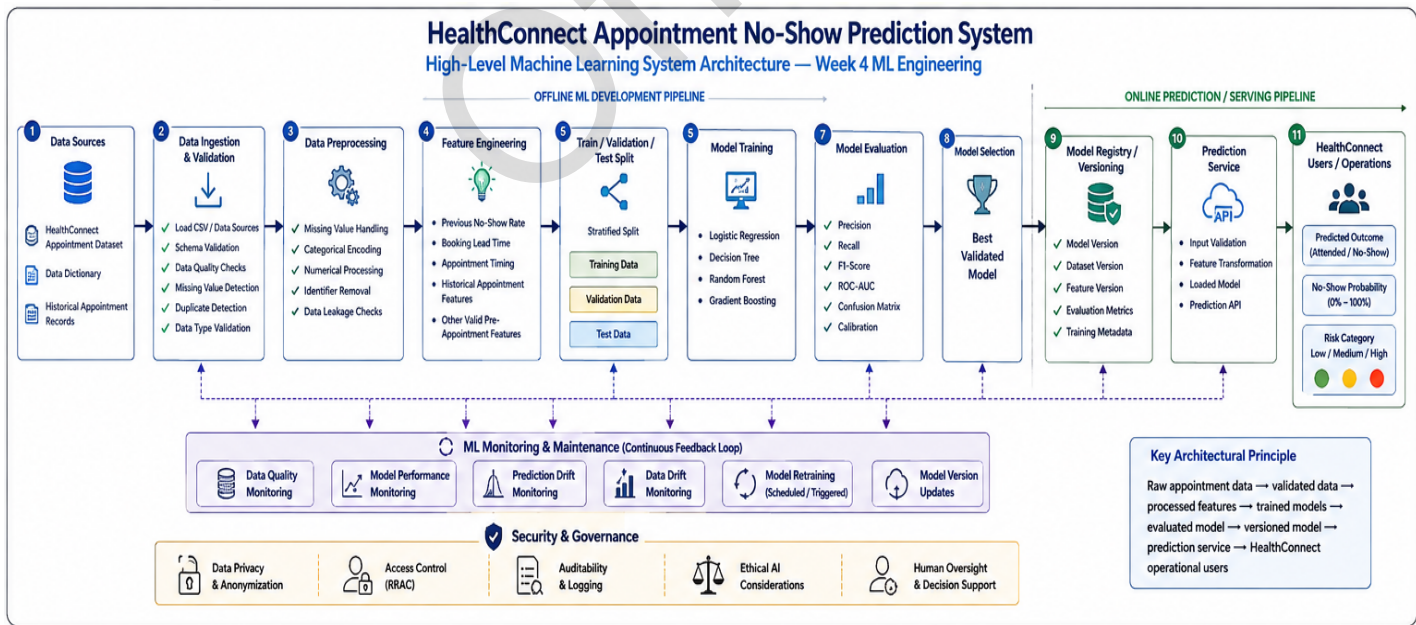
No-Show Probability: 78%

This probability can allow HealthConnect to establish different risk categories.

Probability	Risk Category
0–30%	Low
31–60%	Medium
61–100%	High

The exact thresholds should be validated during later modelling rather than treated as final at Week 4.

3. High-Level System Architecture



4. Proposed ML Workflow

The proposed workflow consists of the following stages.

Stage 1 — Data Ingestion

Load the HealthConnect appointment dataset into the ML environment.

The dataset will initially be stored as a CSV file and loaded using Python/Pandas.

Stage 2 — Data Validation

The dataset will be checked for:

- Missing values
- Duplicate records
- Invalid data types
- Invalid dates
- Impossible ages
- Inconsistent categorical values
- Invalid appointment outcomes
- Historical inconsistencies

For example:

previous_no_shows should not exceed previous_appointments.

Stage 3 — Data Cleaning

The initial dataset contains some missing values.

From the current dataset:

- reminder_channel has missing values.
- distance_to_clinic_km has a small number of missing values.
- waiting_time_minutes has a small number of missing values.

These will be handled using an appropriate strategy rather than simply deleting large portions of the dataset.

For example:

- reminder_channel: missing values may represent appointments where no reminder was sent and therefore could potentially be represented as None/No Reminder.
- Numeric variables such as distance and waiting time may be imputed using an appropriate statistical strategy.
- The chosen approach will be documented and validated.

5. Feature Engineering

Potential engineered features include:

No-show history rate

$\text{previous_no_show_rate} = \text{previous_no_shows} / \text{previous_appointments}$

This may provide more useful information than the raw number of previous no-shows alone.

Appointment lead-time categories

booking_lead_days could potentially be transformed into categories such as:

- Short notice
- Medium notice
- Long notice

Appointment timing

The existing appointment time categories can be encoded for modelling.

Date-related features

Additional features could potentially be derived from appointment_date, such as:

- Month
- Week
- Day of week
- Weekend/weekday

However, only features that would be available at prediction time should be retained.

6. Target Variable

The target variable is:

appointment_outcome

For the initial binary classification system:

No-Show → 1 Attended → 0

Cancelled appointments should be examined during Week 5 before final model training.

A possible alternative approach would be to develop a **three-class classification model**:

Attended No-Show Cancelled

However, the initial system will prioritize **No-Show prediction**, because this directly addresses the primary operational problem.

7. Model Development Strategy

Several classification algorithms can be evaluated.

Baseline

Logistic Regression

Useful because it provides a simple and interpretable baseline.

Tree-based models

Decision Tree

Useful for understanding decision patterns.

Random Forest

Useful for capturing nonlinear relationships and interactions between variables.

Gradient boosting

A boosting model can also be evaluated if appropriate.

The final model should not be selected simply because it has the highest accuracy.

Model selection should consider:

- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion matrix
- Calibration/probability quality
- Interpretability
- Computational requirements

Because missing a patient who is genuinely at high risk of no-show may be operationally important, **recall for the No-Show class** should receive particular attention.

8. Model Evaluation

The evaluation framework will include:

Accuracy

Overall percentage of correct predictions.

Precision

Of the appointments predicted as no-shows, how many actually became no-shows?

Recall

Of the appointments that actually became no-shows, how many did the model identify?

F1-score

Balances precision and recall.

Confusion Matrix

Will show:

Actual Attended No-Show Predicted Attended No-Show

This will make it easier to understand the types of errors the model makes.

9. Key Data Dependencies

The ML system depends on the following data:

1. Appointment records
2. Patient appointment history

3. Previous no-show history
4. Appointment scheduling information
5. Reminder information
6. Appointment timing
7. Distance information
8. Waiting-time information
9. Final appointment outcome for historical training data

The quality of predictions will depend heavily on the completeness and accuracy of these variables.

10. Key Technical Dependencies

The initial implementation is expected to use:

Tool/Library	Purpose
Python	Main programming language
Pandas	Data manipulation
NumPy	Numerical operations
Scikit-learn	ML preprocessing, modelling and evaluation
Matplotlib	Visualization
Seaborn	Exploratory data analysis
Jupyter Notebook	Experimentation and documentation
Git/GitHub	Version control
Joblib	Model serialization

Potential future deployment technologies could include:

- FastAPI for prediction APIs
- Docker for reproducible environments
- Cloud hosting for deployment

These are **future implementation considerations**, not requirements for completing Week 4.

11. Reproducibility Strategy

Reproducibility will be considered from the beginning of development.

The project will use:

Version control

Git and GitHub will be used to track:

- Code
- Notebooks
- Documentation
- Configuration files
- Model-development changes

Random seeds

Random states will be explicitly defined during data splitting and model training where appropriate.

Environment management

Project dependencies will be documented in:

requirements.txt

Example:

pandas numpy scikit-learn matplotlib seaborn jupyter joblib

Data versioning

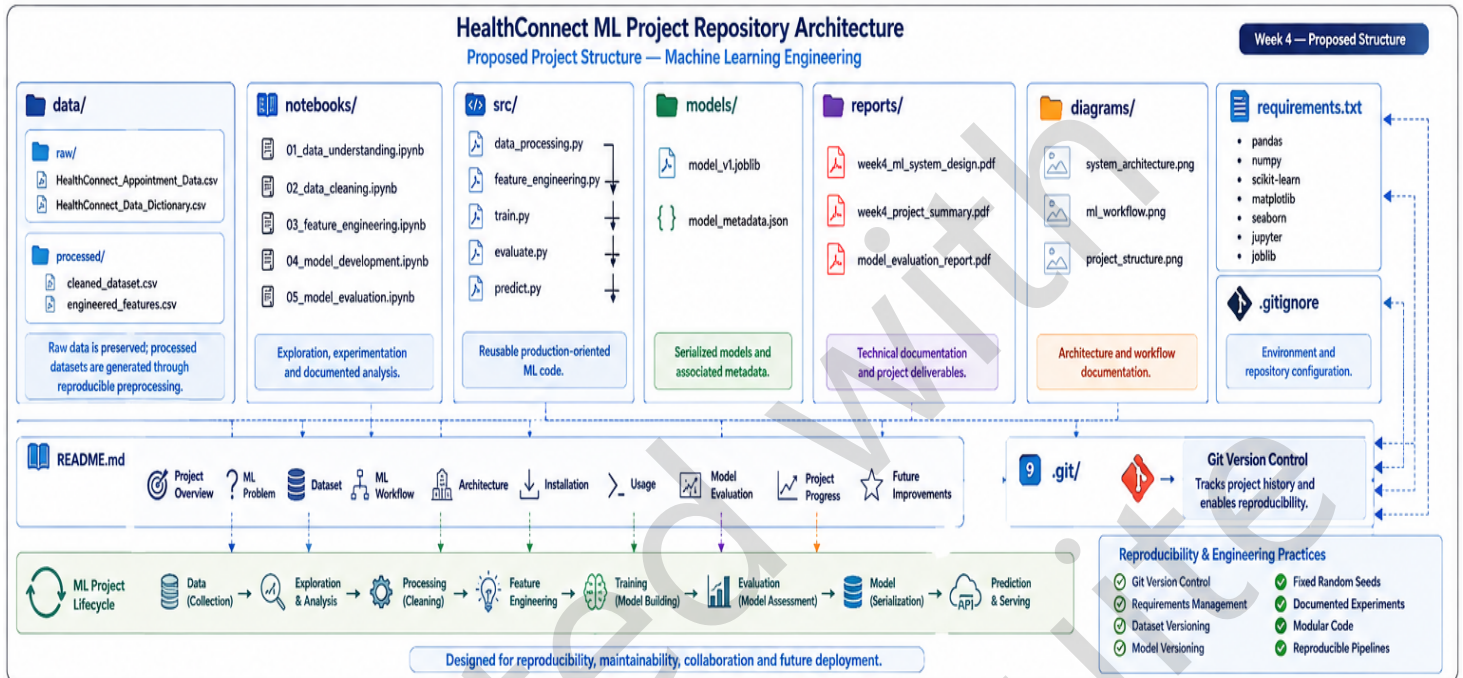
The dataset version used for model development should be documented so that experiments can be reproduced.

Model versioning

Each trained model should have associated information such as:

- Model type
- Dataset version
- Features used
- Training date
- Evaluation metrics
- Model version

12. Proposed Project Structure



PART 3 — ASSUMPTIONS, LIMITATIONS, RISKS AND DEPENDENCIES

13. Assumptions

The following assumptions will guide the initial design:

1. Historical appointment records are representative enough to support initial model development.
2. Appointment information is recorded consistently.
3. Previous appointment and no-show counts are correctly calculated.
4. Information used for prediction will be available before the appointment occurs.
5. The target outcome is correctly recorded.
6. The dataset contains anonymized patient identifiers and does not require personally identifiable information for modelling.

14. Data Limitations

The current dataset has several limitations.

Missing values

Some variables contain missing observations, particularly:

- Reminder channel
- Distance to clinic
- Waiting time

These require appropriate treatment during preprocessing.

Synthetic/anonymized data

The data dictionary identifies the dataset as synthetic/anonymized. Therefore, model performance on this dataset may not represent performance on real HealthConnect patients.

Limited dataset size

The dataset contains approximately **5,000 appointment records**, which is useful for initial experimentation but may not be sufficient to represent all possible real-world patient behaviours.

Historical bias

Historical appointment patterns may contain biases that could be reproduced by the model.

15. Modelling Considerations

Class imbalance

The number of attended appointments may differ significantly from the number of no-shows. Therefore, class distribution must be examined before modelling.

Accuracy alone should not be used as the primary model-selection criterion.

Data leakage

Features that would only become available after the appointment must not be used for prediction.

Overfitting

A model that performs extremely well on training data may perform poorly on unseen appointments.

Train/validation/test separation and appropriate cross-validation will therefore be considered.

16. System Risks

Risk	Potential Impact	Mitigation
Missing data	Reduced model reliability	Appropriate imputation/handling
Class imbalance	Poor no-show detection	Evaluate precision/recall and consider class weighting
Data leakage	Unrealistically high performance	Strict feature availability checks
Overfitting	Poor real-world performance	Validation and cross-validation
Dataset bias	Unfair predictions	Evaluate performance across relevant groups
Poor probability calibration	Incorrect risk interpretation	Calibration analysis
Changing patient behaviour	Model degradation	Future monitoring/retraining
Incorrect input data	Incorrect predictions	Input validation
Privacy concerns	Potential patient harm	Use anonymized/minimized data
Over-reliance on model	Inappropriate interventions	Keep system as decision support

17. Ethical and Safety Considerations

Because this is a healthcare-related system, predictions should be used carefully.

The system should:

- Avoid unnecessary use of personally identifiable information.
- Use anonymized patient identifiers where possible.
- Avoid using predictions to deny healthcare services.
- Avoid treating a prediction as a definitive statement about a patient.
- Monitor performance across demographic groups where appropriate.
- Ensure human oversight when predictions are used operationally.

The model should identify **risk**, not label patients as irresponsible or unreliable.

18. Initial Implementation Approach

The project will initially be implemented using Python and Scikit-learn.

Phase 1 — Data understanding

- Load dataset.
- Review data dictionary.
- Inspect data types.
- Check missing values.
- Check duplicates.
- Examine target distribution.

Phase 2 — Data preprocessing

- Handle missing values.
- Encode categorical variables.
- Process numerical variables.
- Remove identifiers from predictive features.
- Validate date-related features.

Phase 3 — Feature engineering

Create useful predictive variables while avoiding data leakage.

Phase 4 — Dataset splitting

Split the data into training and testing sets.

A stratified split will be considered to preserve the distribution of the target classes.

Phase 5 — Baseline model

Develop a Logistic Regression baseline.

Phase 6 — Model comparison

Evaluate additional models such as:

- Decision Tree
- Random Forest
- Gradient Boosting

Phase 7 — Evaluation

Compare models using:

- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion matrix

Phase 8 — Model selection

Select a model based on both predictive performance and operational suitability.

Phase 9 — Model serialization

Save the selected model using Joblib so it can later be used by a prediction service.

Phase 10 — Future deployment

A future version could expose the model through an API:

HealthConnect Application -- FastAPI -- Preprocessing -- Trained ML Model -- Prediction + Probability

Deployment will be considered after the core model has been validated.